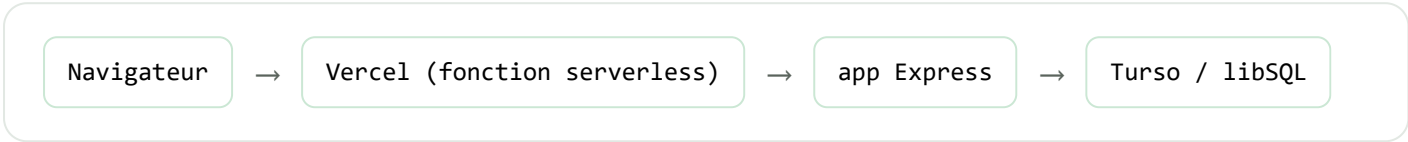


Déploiement sur Vercel

Mise en ligne de l'application « Petit Rayon de Soleil » sur Vercel (hébergement serverless), avec une base de données Turso (SQLite hébergé, compatible serverless).

Hébergeur Vercel (serverless) Base Turso / libSQL Runtime Node.js Prérequis compte GitHub



Pourquoi une base cloud ? Vercel est *serverless* : son système de fichiers est éphémère. Un fichier SQLite local (`soleil.db`) y serait effacé à chaque requête. La base est donc déportée vers **Turso**, qui expose SQLite via le réseau — le code SQL du projet reste identique, seul le connecteur change (`@libsql/client`).

1 Adaptations du code (déjà réalisées)

Le projet a été rendu compatible serverless. Ces changements sont déjà en place dans le dépôt :

Fichier	Adaptation
<code>db.js</code>	Connexion via <code>@libsql/client</code> — Turso si les variables d'environnement sont définies, sinon fichier local. Adaptateur asynchrone conservant l'interface <code>prepare().get/run/all</code> .
<code>auth.js</code>	Fonctions d'accès à la base et middlewares passés en <code>async / await</code> .
<code>server.js</code>	Routes passées en <code>async / await</code> ; gestionnaire d'erreurs global ; l'app n'écoute un port qu'en local.
<code>api/index.js</code>	Point d'entrée serverless : réutilise l'app Express exportée par <code>server.js</code> .
<code>vercel.json</code>	Redirige toutes les requêtes vers la fonction <code>/api</code> .

Les **7 tests automatisés passent** après ces changements (fonctionnement identique en local et en cloud).

2 Créer la base Turso

- 1 Créer un compte gratuit sur turso.tech (connexion via GitHub).
- 2 Créer une base de données (ex. `petit-rayon-de-soleil`).
- 3 Récupérer les deux informations de connexion :

Information	Exemple
Database URL	<code>libsql://petit-rayon-de-soleil-xxxx.turso.io</code>
Auth Token	un long jeton secret (bouton « Create Token »)

3 Initialiser les tables dans Turso

Une seule fois, depuis sa machine, créer les tables et les données de démonstration dans la base cloud :

PowerShell

```
# Renseigner ses vraies valeurs Turso
$env:TURSO_DATABASE_URL="libsql://...turso.io"
$env:TURSO_AUTH_TOKEN="ton-token"
npm run init-db
```

Le message « Base initialisée avec un admin, un utilisateur et 5 messages » confirme la création.

4 Publier le dépôt sur GitHub

Vercel se déploie depuis un dépôt Git. Pousser le projet sur GitHub. Le fichier `.gitignore` exclut déjà `node_modules`, la base locale et les secrets (`.env`).

terminal

```
git init
git add .
git commit -m "Version compatible Vercel + Turso"
git remote add origin https://github.com/<compte>/<repo>.git
git push -u origin main
```

5 Déployer sur Vercel

- 1 Se connecter sur vercel.com via GitHub.
- 2 **Add New** → **Project**, puis sélectionner le dépôt.
- 3 Dans **Environment Variables**, ajouter les deux mêmes clés que pour Turso :

Name	Value
TURSO_DATABASE_URL	libsql://...turso.io
TURSO_AUTH_TOKEN	le jeton secret

- 4 Cliquer sur **Deploy**. Au bout d'environ une minute, Vercel fournit une URL publique, par exemple `https://petit-rayon-de-soleil.vercel.app`.

SÉCURITÉ Le token Turso ne doit **jamais** être commité dans le dépôt. Il est fourni uniquement via les variables d'environnement (locales et Vercel), et `.env` est exclu par `.gitignore`.

6 Comptes de démonstration

Rôle	Email	Mot de passe
Admin	admin@soleil.fr	admin123
Utilisateur	user@soleil.fr	user1234

Procédure de déploiement de « Petit Rayon de Soleil » sur Vercel + Turso · Titre professionnel CDA. Le projet reste exécutable en local (`npm start` → `http://localhost:3000`) sans configuration Turso, grâce au repli sur un fichier SQLite.